Sicurezza dei Sistemi Informativi

Appello d'Esame del 27 Gennaio 2022

Anno Accademico 2021-2022

Politecnico di Torino — Docente: Prof. Cataldo Basile

2022-01-27

1 Testo Ufficiale dell'Esame

Domanda 1: Authenticated Encryption

Spiegare cosa è l'Authenticated Encryption, quali motivazioni hanno portato alla sua introduzione e quali problematiche di sicurezza risolve rispetto all'uso disgiunto di cifratura simmetrica e codici di integrità (MAC). Descrivere brevemente le modalità operative più diffuse (es. GCM, CCM).

Domanda 2: TLS Handshake Protocol

Descrivere dettagliatamente le fasi del protocollo di Handshake di TLS (con riferimento a TLS 1.2 e relative evoluzioni in TLS 1.3). Spiegare come avviene lo scambio di chiavi, l'autenticazione del server (ed eventualmente del client) e come viene garantita la Perfect Forward Secrecy (PFS).

Domanda 3: Principi di Protezione del Software

Spiegare i principi cardine della protezione del software e il modello Man-at-the-End (MATE). Illustrare le principali tecniche difensive, tra cui l'offuscamento del codice, le tecniche di anti-tampering, il watermarking e i limiti intrinseci di tali approcci.

Domanda 4: Standard ISO/SAE 21434

Descrivere che cos'è lo standard ISO/SAE 21434 (Road Vehicles — Cybersecurity Engineering), quali sono i suoi obiettivi principali, come si inserisce nel ciclo di vita dello sviluppo del veicolo e cosa prevede il framework TARA (Threat Analysis and Risk Assessment).

Domanda 5: Autenticazione mediante Crittografia Simmetrica (CRA)

Spiegare il funzionamento dei protocolli di autenticazione Challenge-Response basati su crittografia simmetrica. Evidenziare vantaggi e svantaggi di questo approccio, con particolare riguardo alla gestione e distribuzione delle chiavi segrete, alla scalabilità e alla proprietà di non ripudio.

Domanda 6: Esercizio di Progettazione Firewall

Data una rete aziendale composta da una sottorete interna (Client), una DMZ con server pubblici (Web Server e Mail Server) e l'interfaccia verso Internet, definire la politica di sicurezza e compilare la tabella delle regole di filtraggio per un packet filter stateful nei seguenti scenari:

a. I client interni possono navigare liberamente sul Web (HTTP/HTTPS) verso Internet.

b. Gli utenti da Internet possono raggiungere il Web Server aziendale (TCP 80/443) nella DMZ.

c. Il Mail Server nella DMZ può scambiare posta (SMTP TCP 25) con server di posta esterni su Internet.

d. I client interni possono accedere alla posta aziendale (IMAP su TLS TCP 993) sul Mail Server della DMZ.

e. Nessun host da Internet o dalla DMZ può avviare connessioni dirette verso la sottorete dei client interni.

f. Regola di Default Drop per tutto il traffico non esplicitamente autorizzato.

Domanda 7: Social Engineering: Phishing e Pharming

Definire e confrontare Phishing e Pharming, spiegando i meccanismi di attacco utilizzati dagli avversari (spoofing, DNS cache poisoning, manomissione host file). Indicare un esempio concreto di scenario di Social Engineering e proporre contromisure tecnologiche e organizzative.

2 Soluzione Dettagliata e Guida Risolutiva

Soluzione Domanda 1: Authenticated Encryption (AEAD)

Definizione e Motivazioni: Tradizionalmente, confidenzialità e integrità venivano applicate combinando in modo indipendente un algoritmo di cifratura simmetrica (es. AES in modalità CBC) e un Message Authentication Code (es. HMAC). Questa separazione ha introdotto storicamente gravi vulnerabilità note come *padding oracle attacks* (es. attacco di Vaudenay), soprattutto quando implementata con l'approccio *MAC-then-Encrypt* (utilizzato originariamente in SSL/TLS) o *Encrypt-and-MAC* (SSH). Solo l'approccio *Encrypt-then-MAC* risultava formalmente provabile come sicuro contro attacchi ad injection e chosen-ciphertext (IND-CCA2).
Per eliminare questi errori di composizione e migliorare le prestazioni computazionali, è stato standardizzato il paradigma **AEAD (Authenticated Encryption with Associated Data)**:

- Un singolo algoritmo crittografico gestisce contestualmente sia la cifratura del payload (confidenzialità) sia il calcolo del tag di autenticazione (integrità e autenticità).
- Consente l'inclusione di *Associated Data (AD)*: metadati (come header IP, numeri di sequenza, flag di protocollo) che devono essere autenticati e protetti da manomissione ma trasmessi in chiaro per consentire l'instradamento di rete.

Modalità Principali:

1. **AES-GCM (Galois/Counter Mode)**: Combina la modalità Counter (CTR) per la cifratura a flusso parallela con l'autenticazione tramite moltiplicazione nel campo di Galois GF(2^{128}) (GHASH). È estremamente efficiente e parallelizzabile sia in software che tramite istruzioni hardware dedicate (AES-NI, CLMUL). Richiede che il nonce non venga mai riutilizzato sotto la stessa chiave.
2. **AES-CCM (Counter with CBC-MAC)**: Combina la modalità CTR per la cifratura con un CBC-MAC per l'autenticazione. È sequenziale e richiede due passaggi sul blocco di dati, ma è ampiamente adottato in standard wireless a risorse limitate (es. IEEE 802.11i / WPA2).
3. **ChaCha20-Poly1305**: Stream cipher ChaCha20 combinato con l'autenticatore a una via Poly1305. Ottimale per dispositivi privi di accelerazione hardware per AES.

Soluzione Domanda 2: TLS Handshake Protocol

Fasi di TLS 1.2:

1. **ClientHello**: Il client invia la versione massima supportata di TLS, una sequenza casuale R_C (32 byte), la lista delle ciphersuite supportate (es. `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`) e le estensioni (es. SNI, curve ellittiche supportate).
2. **ServerHello**: Il server risponde selezionando la versione del protocollo, invia il proprio nonce casuale R_S e la ciphersuite prescelta.
3. **Certificate**: Il server invia la propria catena di certificati X.509 per consentire al client di validare la sua identità tramite la catena di fiducia fino a una Root CA trusted.

4. **ServerKeyExchange** (richiesto in DHE/ECDHE): Il server invia i parametri effimeri di Diffie-Hellman ($g, p, g^b \pmod p$ oppure la curva ellittica e il punto pubblico $B = b \cdot G$) firmati con la propria chiave privata (es. RSA o ECDSA). Questa firma garantisce l'autenticità dei parametri effimeri.

5. **ServerHelloDone**: Notifica la fine della fase di negoziazione iniziale da parte del server.

6. **ClientKeyExchange**: Il client verifica la firma del server, valida il certificato, genera il proprio parametro effimero (g^a oppure $A = a \cdot G$) e lo trasmette al server. Entrambe le parti calcolano il segreto condiviso *Pre-Master Secret* ($PMS = g^{ab} \pmod p$).

7. **Derivazione Chiavi**: Tramite una Pseudo-Random Function (PRF) basata su HMAC-SHA256, viene calcolato il *Master Secret*:

$$MS = \text{PRF}(PMS, \text{"master secret"}, R_C \parallel R_S)$$

   Dal Master Secret vengono derivate le chiavi di sessione simmetriche per client e server (chiavi di cifratura, IV e MAC).

8. **ChangeCipherSpec & Finished**: Entrambe le parti si scambiano il messaggio crittografato `Finished`, che contiene un hash verificato di tutti i messaggi scambiati nell'handshake, prevenendo attacchi di man-in-the-middle e downgrade attack.

Perfect Forward Secrecy (PFS): L'uso di chiavi effimere (DHE / ECDHE) garantisce che la compromissione futura della chiave privata a lungo termine del server (es. la chiave RSA usata per il certificato) non permetta a un attaccante di decifrare il traffico storicamente registrato, poiché i parametri effimeri a e b vengono distrutti dalla memoria volatile al termine della sessione.
Differenze con TLS 1.3: In TLS 1.3 il round-trip time viene ridotto da 2-RTT a 1-RTT (o 0-RTT con Early Data). Vengono rimosse le ciphersuite statiche e obsolete (RSA static key exchange, CBC, RC4, 3DES, SHA-1). L'algoritmo di scambio chiavi è obbligatoriamente effimero (ECDHE/DHE), integrando la generazione chiavi fin dal primo messaggio (`ClientHello key_share`).

Soluzione Domanda 3: Protezione del Software e Modello MATE

Il Modello Man-at-the-End (MATE): A differenza del classico modello di rete Man-in-the-Middle (in cui l'attaccante intercetta un canale tra due endpoint fidati), nel modello MATE l'attaccante ha il possesso fisico e il controllo totale della piattaforma su cui il software viene eseguito. L'attaccante può agire con privilegi di amministratore/root, utilizzare debugger (GDB, x64dbg), disassemblatori/decompiler (IDA Pro, Ghidra), monitorare la memoria, modificare registri e file binari.
Tecniche Difensive:

- **Offuscamento del Codice**: Trasforma il codice sorgente o binario in una forma equivalente dal punto di vista semantico ma estremamente complessa da comprendere e analizzare.
  - *Control Flow Flattening*: Trasforma la struttura a blocchi e cicli in un unico grande switch governato da una variabile di stato cifrata.

  - *Predicati Opachi (Opaque Predicates)*: Inserimento di rami condizionali il cui esito logico è staticamente noto al compilatore ma indecidibile per un analizzatore statico.
  - *String & Instruction Encryption*: Cifratura di stringhe critiche e frammenti di codice decifrati solo al momento dell'esecuzione (just-in-time).

- **Anti-Tampering**: Meccanismi di auto-controllo dell'integrità dell'eseguibile (es. calcolo di checksum a runtime della sezione `.text`), rilevamento di breakpoint software (`0xCC`), controlli anti-debugging (es. flag `IsDebuggerPresent`, `ptrace(PT_DENY_ATTACH)`).

- **Software Watermarking**: Inserimento di una firma o identificativo univoco nascosto nella struttura logica o nei pattern di istruzioni del software per tracciare la proprietà intellettuale o identificare la sorgente di leak illegali.

- **White-Box Cryptography**: Tecniche matematiche per incorporare chiavi crittografiche direttamente all'interno delle tabelle di lookup e trasformazioni dell'algoritmo (es. White-Box AES), evitando che la chiave possa essere estratta leggendo la memoria.

Limite fondamentale: Nessuna tecnica puramente software può garantire protezione assoluta nel modello MATE (teorema di Barak et al. sull'impossibilità dell'offuscamento virtual black-box perfetto); l'obiettivo ingegneristico è aumentare il costo e il tempo dell'attacco fino a renderlo non economicamente conveniente.

Soluzione Domanda 4: Standard ISO/SAE 21434

Inquadramento e Obiettivi: Lo standard **ISO/SAE 21434** (Road vehicles — Cybersecurity engineering) è lo standard di riferimento internazionale che disciplina l'ingegneria della sicurezza informatica per i veicoli stradali (automotive). Il suo obiettivo primario è garantire che la sicurezza informatica sia considerata come un requisito ingegneristico integrato lungo tutto il ciclo di vita del veicolo: dalla concezione, progettazione, produzione, funzionamento e manutenzione (OTA updates), fino al decommissioning.
Il Framework TARA (Threat Analysis and Risk Assessment): È la metodologia prescritta dallo standard per identificare e quantificare i rischi di sicurezza informatica. Si articola nei seguenti passi:

1. *Asset Identification*: Identificazione degli asset (ECU, bus CAN, gateway telematici, sensori ADAS) e delle relative proprietà di sicurezza (Confidenzialità, Integrità, Disponibilità, Autenticità).

2. *Threat Scenario Identification*: Mappatura dei possibili scenari di minaccia (es. iniezione di messaggi spoofati sul bus CAN tramite porta OBD-II o infotainment compromesso).

3. *Impact Rating*: Valutazione delle conseguenze dell'attacco secondo i criteri SFOP (Safety, Financial, Operational, Privacy).

4. *Vulnerability Analysis & Attack Path Analysis*: Identificazione delle vulnerabilità e dei percorsi di attacco necessari per sfruttarle.

5. *Attack Feasibility Rating*: Valutazione della fattibilità dell'attacco basata su tempo richiesto, competenze specialistiche, conoscenza del target, finestre di opportunità e attrezzature necessarie.

6. *Risk Determination & Treatment*: Calcolo del livello di rischio complessivo e determinazione della strategia di mitigazione (riduzione, accettazione, trasferimento, evitamento).

Soluzione Domanda 5: Autenticazione Challenge-Response Simmetrica (CRA)

Funzionamento del Protocollo:

1. L'entità verificatrice V (es. server) genera un valore non predicibile R (*nonce* o challenge casuale) e lo invia all'entità richiedente C (client).
2. C calcola la risposta crittografica applicando una funzione F con la chiave segreta condivisa K:

$$\text{Response} = F(K, R) \quad \text{oppure} \quad \text{Response} = \text{HMAC}_K(R) \quad \text{oppure} \quad E_K(R)$$

3. C invia la risposta a V.
4. V calcola autonomamente il valore atteso $F(K, R)$ e lo confronta con la risposta ricevuta. Se coincidono, l'identità di C è autenticata.

Vantaggi:

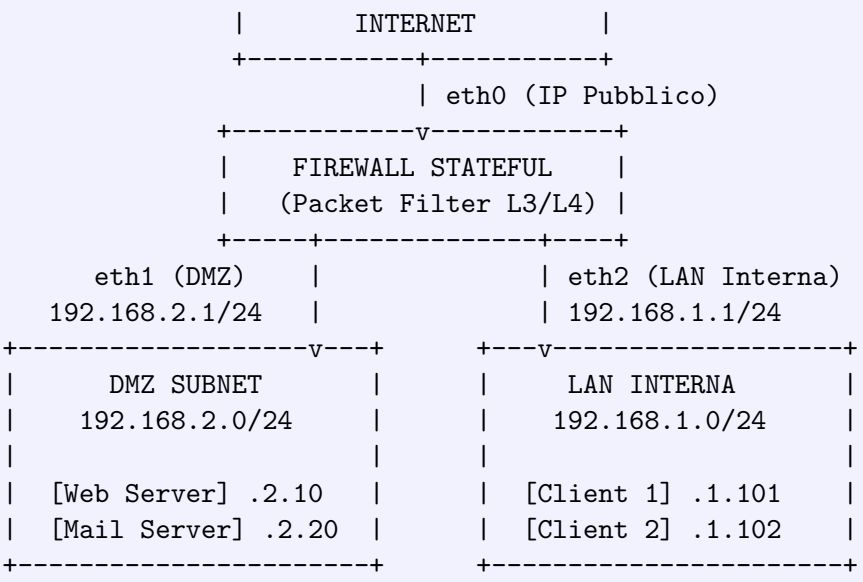
- *Efficienza computazionale*: Si avvale di primitive simmetriche (AES, SHA-256, HMAC), molto più veloci rispetto alle operazioni asimmetriche a chiave pubblica (RSA, ECC).
- *Resistenza al Replay Attack*: Il nonce fresco generato ad ogni sessione impedisce all'attaccante di riutilizzare pacchetti intercettati in precedenza.
- La password o chiave a lungo termine non viaggia mai in chiaro sul canale di comunicazione.

Svantaggi e Limiti:

- *Distribuzione e Gestione delle Chiavi*: Ciascuna coppia di entità necessita di una chiave segreta distinta. In una rete con N nodi, sono necessarie $\frac{N(N-1)}{2} = O(N^2)$ chiavi condivise, creando problemi di scalabilità a meno di non introdurre un centro di distribuzione di fiducia (KDC come in Kerberos).
- *Compromissione del Verifier*: Il server deve memorizzare la chiave segreta K di tutti gli utenti in chiaro (o in forma reversibile). La compromissione del database del server compromette immediatamente l'intero sistema.
- *Assenza di Non-Ripudio*: Poiché entrambe le parti possiedono la stessa chiave K, V non può dimostrare a una terza parte che un messaggio sia stato originato univocamente da C, poiché V stesso avrebbe potuto forgiarlo.

Soluzione Domanda 6: Regole del Firewall (Packet Filter Stateful)

Politica di Sicurezza e Architettura DMZ (Screened Subnet): Si adotta una configurazione a tre zone (*Screened Subnet*) mediata da un firewall stateful perimetrale:



Subnetting IP e Instradamento:

- **Interfaccia Esterna (`eth0`)**: Collegata al provider Internet (`0.0.0.0/0`). Applica Source NAT (SNAT/Masquerade) per il traffico originato dai client interni e Destination NAT (DNAT/Port Forwarding) per il Web Server esposto.
- **DMZ (`eth1`, `192.168.2.0/24`)**: Gateway `192.168.2.1`. Ospita i server pubblici isolati: Web Server (`192.168.2.10`) e Mail Server (`192.168.2.20`).
- **LAN Interna (`eth2`, `192.168.1.0/24`)**: Gateway `192.168.1.1`. Ospita i client riservati. Non è direttamente instradabile né raggiungibile dall'esterno.

Tabella delle Regole del Firewall (Default Policy: DROP):

ID	Dir	Src IP	Dst IP	Proto	Dst Port	Stato	Azione & Motivazione
1	Any	Any	Any	Any	Any	ESTABLISHED, RELATED	**ACCEPT** Risposte a connessioni lecite
2	In/Fwd	192.168.1.0/24	Any (Internet)	TCP	80, 443	NEW	**ACCEPT** Navigazione Web client interni
3	In/Fwd	Any (Internet)	192.168.2.10	TCP	80, 443	NEW	**ACCEPT** Accesso pubblico al Web Server DMZ
4	In/Fwd	192.168.2.20	Any (Internet)	TCP	25	NEW	**ACCEPT** Mail server DMZ invia posta esterna
5	In/Fwd	Any (Internet)	192.168.2.20	TCP	25	NEW	**ACCEPT** Mail server DMZ riceve posta

Soluzione Domanda 7: Phishing, Pharming e Social Engineering

Definizioni e Meccanismi:

- **Phishing**: Attacco di ingegneria sociale condotto principalmente tramite email, messaggistica istantanea o SMS (Smishing), in cui l'attaccante impersona un'entità fidata (banca, fornitore di servizi, amministrazione aziendale). Il messaggio contiene tipicamente un senso di urgenza e induce la vittima a cliccare su un link fraudolento (che conduce a un sito di spoofing clone) o ad aprire un allegato infetto (malware).
- **Pharming**: Attacco più sofisticato e trasparente alla vittima, basato sulla manomissione dei meccanismi di risoluzione dei nomi (DNS). L'attaccante reindirizza il traffico destinato a un dominio legittimo (es. `banca.it`) verso l'indirizzo IP di un server malevolo controllato dall'attaccante, anche se la vittima digita correttamente l'URL nella barra degli indirizzi del browser. Può essere perpetrato tramite:
  - *DNS Cache Poisoning* (es. sfruttando la vulnerabilità di Kaminsky sui server DNS resolver ricorsivi);
  - Compromissione del router domestico/aziendale con modifica dei DNS server assegnati via DHCP;
  - Modifica locale del file `hosts` della macchina della vittima mediante malware.

Contromisure Efficaci:

- *Tecnologiche*: Autenticazione email forte (SPF, DKIM, DMARC), implementazione di DNSSEC per garantire l'autenticità e integrità delle risposte DNS, certificati digitali e protocollo TLS con controllo HSTS (evita che l'utente accetti certificati falsi), autenticazione a più fattori (MFA con token FIDO2/WebAuthn resistente al phishing).
- *Organizzative/Umane*: Programmi continui di Security Awareness Training e campagne di simulazione di phishing periodiche.

Alternative Implementation Note: Concorrenza e Strutture Dati Rust

Nello sviluppo di software di rete sicuro (es. proxy di autenticazione, demone per packet filtering o handler di handshake crittografici), l'approccio idiomatico in Rust prevede l'uso di primitive di concorrenza standard per evitare race condition, data race e garantire la thread-safety garantita a tempo di compilazione:

- `std::sync::mpsc::channel`: Canali di comunicazione multi-producer single-consumer per l'invio asincrono di eventi di sicurezza dai worker thread al log collector o al motore di correlazione eventi.
- `std::sync::RwLock`: Per strutture dati prevalentemente lette (es. tabella delle regole del firewall, whitelist di indirizzi IP, session cache di TLS) che richiedono letture parallele concorrenti senza lock e scrittura esclusiva solo durante gli aggiornamenti.
- `std::sync::atomic` (`AtomicUsize`, `AtomicBool`): Per contatori di performance, metriche di rate-limiting e flag di shutdown senza overhead di bloccaggio del mutex.
- `std::sync::Arc`: Puntatore a conteggio di riferimenti atomico per condividere contesti di crittografia e trust store X.509 tra thread multipli.